

3-1 · Pandas를 활용한 데이터 처리 · 실습 1.5H

선택·필터·변환

고르고, 바로잡고, 파생 열을 만들어 저장까지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① 열 하나(Series)·여럿(DataFrame), `iloc`·`loc` 로 행을 골라낸다
- ② 불리언 인덱싱과 `loc` 로 조건과 열 목록을 한 번에 지정한다
- ③ `str.strip()`·`to_datetime`·`assign` 으로 타입을 바로잡고 파생 열을 만든다
- ④ 결측 처리 세 방법을 실행하고 `orders_step1.parquet` 를 저장한다

2단계에서 `strip` 을 일부러 빼고 세어 봅니다 — 20건이 조용히 사라지는 것을 수치로

여덟 단계 — orders.csv 에서 step1.parquet 까지

1~8단계는 transform.py 한 파일에 이어 붙인다 — 4단계부터 df 가 바뀌니 순서를 건너뛰지 않는다

1. 고르기	2.3. 거르기	4.5. 변환	6. 결측	7.8. 저장
열.행 iloc·loc	조건 필터 unique·counts	to_datetime assign·dt	dropna fillna·mean	Parquet 미니 과제

고르기와 거르기 — 핵심 줄

하려는 일	코드	결과
열 하나	df['price']	Series
열 여럿	df[['order_id', 'price']]	DataFrame — 대괄호 두 겹
위치로	df.iloc[0:3]	3행 — 끝 제외
이름표로	df.loc[0:2]	3행 — 끝 포함
조건 필터	df[(조건1) & (조건2)]	big — 고액·다수량
조건 + 열 목록	df.loc[isin([1, 2, 3]), [열 3개]]	sub — 3개 열만

확인 포인트: iloc[0:3] 과 loc[0:2] 가 둘 다 3행 — loc 은 끝을 포함하기 때문 · sub 에는 지정한 3개 열만

strip 없이 '식품' 을 세면 — 오류 없이 20건이 빠진다

눈으로는 똑같아 보이는 '식품' 이 다른 값으로 취급됩니다 — 실무에서 집계가 미묘하게 안 맞는 흔한 원인

1. strip 없이

```
df['category']  
== '식품'
```

2. strip 하고

```
.str.strip()  
== '식품'
```

3. 건수 비교

```
농천 건수  
= 20
```

4. 왜?

'식품' 은
다른 값

5. 습관

문자열 열은
받자마자 strip

고유값과 분포 — 오염을 눈에 보이게

1 `df['category'].unique()`
'식품' 과 ' 식품 ' 이 따로 나온다

2 `df['category'].nunique()`
5 가지로 세어지지만 실제 카테고리는 4 가지

3 `df['category'].value_counts()`
값마다 건수 — 두 줄로 갈린 것이 그대로 보인다

실제 카테고리

4 가지

≠ 5 가지

공백 오염은 오류를 내지 않아 가장 잡기 어렵다 — 3-5 품질 리포트에서 이 항목을 자동 검사로 만든다

타입을 바로잡고 파생 열 · 날짜 부품

공백 제거	<code>df['category'] = df['category'].str.strip()</code>
날짜 변환	<code>df['order_date'] = pd.to_datetime(df['order_date'])</code>
파생 열	<code>df = df.assign(amount=df['price'] * df['quantity'])</code>
날짜 부품	<code>df['order_date'].dt.year · .dt.month · .dt.day_name()</code>
날짜 필터	<code>df[df['order_date'] < '2026-06-08']</code> — 6월 첫 주 주문

확인 포인트: `order_date` 가 `datetime64[us]` · `amount` 열 추가 · 카테고리 4가지 — 문자열인 채로는 `.dt` 자체를 쓸 수 없다

결측을 다루는 세 방법 — 오늘은 방법만

방법	코드	결과
① 버린다	<code>df.dropna(subset=['price'])</code>	결측 행 제거 — 985 행
② 채운다	<code>fillna(df['price'].median())</code>	중앙값으로 — 결측 0
③ 그대로 둔다	<code>df['price'].mean()</code>	집계에서 자동 제외 (기본)

`df.isna().sum()` — 결측은 price·amount 각 30건(amount 는 price 에서 파생)

「몇 건을 빼고 낸 평균인지」 모르면 위험 · 0 으로 채우면 「값을 모름」이 「값이 0」이 되어 평균 단가가 낮아진다 — 실제 결정은 3-5 에서

같은 날짜 열을 두 형식으로 읽으면

CSV 로 읽은 order_date

타입을 저장하지 않아 다시 읽으면 문자열

- dtype → str (pandas 2: object)
- 매번 to_datetime 이 필요
- .dt 를 못 쓴다

VS

Parquet 로 읽은 order_date

dtype 이 파일에 함께 저장된다

- dtype → datetime64[us]
- amount·year·month·weekday 도 그대로
- 3-2 에서 바로 쓴다

이 1.5시간이 끝날 때 확인할 것

- 1 `df['price']` 는 Series, `df[['order_id', 'price']]` 는 DataFrame
- 2 strip 없이 세면 20건이 빠진다 — 놓친 건수 20
- 3 `unique()` 에 '식품' 과 ' 식품 ' 이 따로 나온다
- 4 `order_date` 가 `datetime64[us]`, `amount·year·month·weekday` 추가
- 5 `orders_step1.parquet` 를 다시 읽은 `order_date` 가 `datetime64[us]`
- 6 미니 과제: 카테고리 4가지 평균 · quantity 5 · amount 상위 10건

3-1 · 실습 1.5H 마무리

다음은 정리·회고 (0.5H)

오늘 쓴 Pandas 문법을 한 장으로 모으고, 3-2 가 쓸 orders_step1.parquet 를 확인합니다

✓ Pandas 요약표 — 읽기부터 저장까지

✓ ls -lh 로 파일 네 개 확인

✓ 자주 겪는 오류 네 가지

✓ 3-2 Pandas 데이터 분석 예고

가져갈 것

orders_step1.parquet · (1015, 10) 안팎